CSIS 4495: Applied Research Project

Progress Report 1

Student: Hoang (Chris) Nghiem Dac

Student ID: 300416011

Course: CSIS 4495 – 002

Project: PowerShell-Based Active Directory Security Auditing Tool with a Local Plain-Language Remediation Dashboard

Repository: https://github.com/Ha0n9/F26_4495_S2_ChrisN

Reporting period: September 27 to October 12, 2026

1. Work Date and Hours Log

Date	Hours	Description of work done
Sep 27, 2026	1	Created the Github repository; committed the proposal to DocumentsAndReports
Sep 27, 2026	1	Revised the proposal after review
Sep 28, 2026	5	Designed the JSON contract between the PowerShell module and the dashboard, defined the finding schema (checkId, severity, affectedObjects, evidence) and wrote CONTRACT.md
Sep 29, 2026	4	Built the finding dictionary as a separate content layer, so detection logic and plain language remediation stay independent, wrote entries for the first three checks
Sep 30, 2026	6	Evaluated Ant Design against shadcn/ui for the dashboard and chose Ant Design for its ready-made data table, severity tags and expandable rows; scaffolded the Vite, React and TypeScript project

Oct 1, 2026	2	Built the dashboard data layer: TypeScript types mirroring the JSON contract, and the join between findings and dictionary entries on checkId
Oct2, 2026	5	Built dashboard components for findings display, committeed to repo
Oct 3, 2026	1.5	Snapshotted the AD lab VM and reviewed the existing Windows Server 2022 domain; planned the OU structure needed for seeded test accounts
Oct 4, 2026	2	Created test OUs, users and groups through PowerShell rather than the GUI, to build familiarity with the ActiveDirectory cmdlets
Oct 5, 2026	3.5	Researched Get-ADUser properties; confirmed that LastLogonDate and whenCreated are not returned unless requested with Properties
Oct 6, 2026	3	Investigated lastLogonTimestamp replication and found it updates only every 9 to 14 days by default, which makes staleness detection approximate; recorded this as a limitation to state in the report
Oct 7, 2026	3	Tested lowering and restoring the default domain password policy, and confirmed the change was reversible before relying on it
Oct 8, 2026	2	Built the nested group structure (Domain Admins, IT support, help desk) and compared Get-ADGroupMember with and without -Recursive; found that -Recursive flattens membership but discards the path
Oct 9, 2026	4	Researched resolving privileged groups by SID rather than by name, after finding that group names are localized and a name-

		based lookup would silently return nothing on a non-English domain
Oct 10, 2026	2.5	Reviewed the progress report template, reconstructured the repository into Implementation and DocumentsAndReports folders, added the repository README.
Oct 10, 2026	4	Wrote Initialize-VulnerableLab.ps1, which seeds know bad AD configurations and writes ground-truth.json for detection accuracy measurement
Oct 12, 2026	1	GitGuardian flagged a hardcoded password in the lab seeding script, rewrote it to generate per-account random passwords using a cryptographic RNG
Oct 12, 2026	1	Purged the secret from git history with git filter-repo and force pushed, verified with git log -S that no commit still contains it
Oct 12, 2026	2	Committed the ADSecurityAudit module structure: manifest, loader, finding builder, and the aggregator that produces the JSON scan envelope
Oct 12, 2026	0.5	Commited the weak domain password policy check, the first implemented detection check
Oct 12, 2026	2	Wrote Pester unit tests: a passing suite for the password policy check and specification suites for two unimplemented checks
Oct 12, 2026	1	Wrote the lab setup runbook covering seeding, verification and the first end-to-end scan

2. Summary Description of Work Done

   Scope and architecture decisions:

- Moved away from the original mini-SOC idea (a public honeypot feeding a SIEM) because the result would have been an environment dependent setup rather than a self-contained tool another administrator could install, which is what the course requires
- Setteled on a three layer architecture: a PowerShell module that only detects and emits raw JSON, a separate finding dictionary holding the plain language explanation and remediation, a local React dashboard that joins the two on checkId
- Dropped the SQLite database named in the proposal. Timestamped JSON files are sufficient for the current scope, and a persistence layer would add maintenance cost without serving a required feature
- Decided the dashboard runs on localhost only, because scan output contains live account names, group structure and delegation paths

Implementation

- Dashboard: Vite, React, TypeScript in strict mode, ant Design components. Three views: findings, coverage, raw JSON. Finding are grouped by severity, each expanding to show the plain language explanation, remediation steps, affected objects and raw evidence
- Designed the coverage view specifically to separate ‘this check found nothing’ from ‘this check could not run’. A security tool that reports a clean result about ground it never examined is worse than one that reports nothing at all
- PowerShell module: manifest, loader, a finding builder that defines the JSON contract in one place, and an aggregator that runs each check inside its own try/catch so one broken check cannot end the scan

- Implemented the first detection check, the weak domain password policy check, with a configurable baseline rather than hardcoded thresholds
- Wrote the lab seeding script, which creates the known-bad configurations and writes ground-truth.json recording exactly what it seeded. That file is what detection accuracy will be measured against
- Wrote Pester unit tests. The implemented check has a passing suite. The two unimplemented checks have specification suites that currently fail, describing what each must do before it is written

Issues encountered and how they were handled

- GitGuardian flagged a hardcoded password committed in the lab seeding script. Rewrote the script to generate a separate random password per account using a cryptographic RNG rather than Get-Random, with nothing written to disk, then purged the literal from the git history with git filter-repo and verified with git log -S that no commit still contained it
    - Regarding the Git history: Modifying the file doesn't remove that string from the history. It's still in commit d1***bb. I had two options: 1) continue editing forward without rewriting the history. That string was never a real credential of any system, just a password for an isolated lab account, so the actual risk is almost negligible. The repository is also being monitored by the teacher, so rewriting the history would change all the commit hashes and look more suspicious than useful. 2) Rewrite the history using `git filter-repo` and then force push. A small repository, one person, few commits, so this is feasible. But if the teacher has cloned it, the version on their machine will be different.

I chose to overwrite using `git filter-repo`. The commit hash will change, but previous commits retain their hashes because the file doesn't exist yet, and that commit will become an orphan commit. The hash can still be accessed if you know it. And if you want to completely remove it, you need to contact Github Support. But isolating a lab password isn't worth it. However, a force push might be necessary to avoid completely erasing the data. With a version already cloned to the user's machine, using git pull will report a conflict error.

- Repository check-ins were sparse between October 3 and 9 because that period was local AD lab work and research rather than code. Going forward, research notes will be committed as markdown so the repository reflects the work even on days with no code
- DNS was never pointed at a public resolver (e.g 8.8.8.8) on either machine, a public DNS server has no knowledge of **corp.local** and breaks AD service discovery
- Domain join failed intermittently. Usually cause by misconfiguration, time drift, firewall blocking Kerberos/LDAP. But has been discovered with ping and nslookup that the problem is because of DNS misconfiguration

Reseach findings worth recording

- lastLogonTimestamp replicates only every 9 to 14 days by default, so staleness detection is approximate rather than exact. The accurate attribute, lastLogon, is not replicated and would require querying every domain controller
- Privileged group names are localized, so Domain Admins must be resolved by its RID of 512 rather than by name for the tool to work on a domain other than the lab. Portability across domains is one of the proposal's evaluation criteria

- Get-ADGroupMember -Recursive flattens nested membership but discards the membership path, which the dashboard needs in order to tell an administrator which group to edit

Changes to the proposal

- The proposal listed a local SQLite database as the likely storage option. Replaced with timestamped JSON files, SQLite remains an optional extension if time allows

Plan for the next period

- Stand up the lab, run the seeding script, and verify the seeded misconfigurations by hand
- Implement the stale accounts and excessive privileged membership checks against their test specifications
- Run the module against the lab and compare the output to ground-truth.json
- Connnect real module output to the dashboard, replacing the sample data, for midterm demo

3. Repo Check in of implementation completed

   The files and folders I have checked into the repository are as follows:

   DocumentsAndReports/

   - HoangN_Proposal.pdf – the approved project proposal

   Implementation:

   - Ad-audit-dashboard/
     - Vite, React and TypeScript project. Source under src/, split into types, lib, components and theme
       - Types/ holds the TypeScript half of the JSON contract
       - Lib/ holds the join, scoring and formatting logic as pure functions, with no React and Ant Design imports, so it can be unit tested on its own

        - Components/ holds one component per region of the page
    - CONTRACT.md – the JSON contract between the module and the dashboard
    - Public/data/finding-dictionary.json – the plain language content layer
    - Public/data/scan-result.sample.json, sample data for development. Real scan output is gitignored because it contains live domain detail
- ADSecurityAudit
    - ADSecurityAudit.psd1 and .psm1, the module manifest and loader
    - Public/Invoke-ADSecurityAudit.ps1, the aggregator and JSON envelope
    - Private/New-AuditFinding.ps1, which builds finding objects to the contract
    - Private/Checks/Test-WeakPasswordPolicy.ps1, implemented
    - Private/Checks/Test-StaleEnabledAccount.ps1 and Test-ExcessiveAdminMembership.ps1, stubs carrying the research notes for each
    - Tests/, Pester unit tests with stand-ins for the ActiveDirectory cmdlets so the suite never touches a domain
    - README.md documenting the layout, the design rules and how to add a check
- Lab
    - Initialize-VulnerableLab.ps1, the seeding script with ground truth output
    - LAB-SETUP.md, the runbook from checkpoint to first end-to-end scan

README.md at the repository root, describing the layout and how the two halves fit together.

4. AI Use section

4.1 Table of AI Tools and Specific Use

AI Tool Name	Version, Account Type	Specific feature for which the AI tool was used	Value Addition
Claude (Anthropic)	Opus 5 (Medium), Pro	Generates the frontend interface based on the user's technical specifications. Acts as a technical advisor when the user needs to search for or seek advice.	AI has saved me a lot of time compared to traditional methods. It also allows one person to multitask within the same amount of time. Previously, when working on freelance or corporate projects, I would spend a lot of time going through the SDLC (Software Development Life Cycle) steps: gathering client requirements, analyzing, evaluating, proposing, and planning, before

			starting the demo development phase. Now, it saves me a significant amount of time in the demo development phase because I already have complete technical specifications. For example, I can directly specify what information I need in the dashboard, how it should receive input, how the information should be presented, which design standards to use, and whether to use third-party libraries to standardize the interface professionally. Instead of choosing all the technologies and libraries and starting from

			scratch, I can directly ask the AI to build the frontend according to my requirements, verify its feasibility, and adjust it to my personal needs to suit the project.
GitGuardian	Free Github integration	Automated secret scanning on the Github repository. Flagged a hardcaoded password committed in the lab seeding script	I'm not sure if this is a generative tool or just a scanner of github. But it has suggestion so I will classify it as an AI scanner tool. It helps me scan all the code I push to GitHub with clear warnings and tags categorized by severity level. It serves as both a protective layer and an important warning system, especially for real-world company or corporate projects. It allows me to quickly

			detect errors that might otherwise be overlooked during the work process due to a lack of professional quality control

4.2 Appendix: Prompt History

(Original prompt is in Vietnamese, but I will translate it to English so instructor can follow up)

Prompt 1:

Ok, tạm thời là biết thế. Chốt thì giữa 2 bên sẽ giao tiếp bằng JSON. Đó sẽ là chuẩn trung từ bây giờ trở đi. Với giao diện frontend. Yêu cầu đặt ra là thể hiện được càng nhiều thông tin càng tốt. Nếu có thể sử dụng các thư viện có sẵn như shacdn hay ant design. Nhớ kỹ, *tuyệt đối không tự tạo các design cho components*

Translate:

Okay, that's all for now. Basically, the two sides will communicate using JSON. That will be the standard from now on. For the frontend interface, the requirement is to display as much information as possible. If possible, use readily available libraries like SHADCN or AntDesign. Remember, *absolutely do not create your own designs for the components*.

Answer:
Rõ. JSON là chuẩn trung, và frontend không tự thiết kế component nào.

Về thư viện: tao khuyên Ant Design, không phải shadcn.

Lý do nằm đúng ở ràng buộc ông vừa đặt ra. shadcn/ui về bản chất là các primitive được copy thẳng vào repo của ông rồi tự compose lại. Cái Table của nó là headless, muốn có sort và filter thì phải gắn thêm TanStack Table và tự dựng layout. Tức là ông vẫn phải ra quyết định thiết kế, chỉ là ở tầng thấp hơn. Điều đó ngược với thứ ông muốn.

Ant Design thì ngược lại, nó cho sẵn nguyên bộ component đúng kiểu dashboard dữ liệu dày:

- Table có sẵn sort, filter, pagination, expandable row (dùng cho phần technical details của mỗi finding)
- Statistic và Card cho khối summary ở đầu trang
- Tag với preset color cho severity
- Descriptions để đổ evidence dạng key-value mà không cần tự căn layout
- Collapse, Alert, Progress cho health score
- Badge cho số đếm

Gần như toàn bộ màn hình demo ghép được từ component có sẵn, ông chỉ viết logic join dữ liệu. Đó là điểm mạnh với dự án này, vì quỹ thời gian học cái mới nên dồn cho PowerShell chứ không phải cho frontend.

Một lưu ý nhỏ: nếu ông muốn “thể hiện càng nhiều thông tin càng tốt”, Table với expandable row sẽ hiệu quả hơn là layout dạng card. Card đẹp nhưng tốn diện tích, mỗi màn hình chỉ hiện được ít finding. Table cho phép nhìn thấy nhiều dòng cùng lúc, lọc theo severity, rồi bung ra xem chi tiết khi cần. Cách này cũng đúng với cách các tool audit thật trình bày kết quả.

Translate:

Clearly. JSON is the middle standard, and the frontend doesn't design any components itself.

Regarding libraries: I recommend Ant Design, not shadcn.

The reason lies precisely in the constraints you just imposed. shadcn/ui is essentially primitives copied directly into your repository and then automatically composed. Its tables are headless; to have sort and filter, you need to add TanStack Table and build the layout yourself. This means you still have to make design decisions, just at a lower level. That's the opposite of what you want.

Ant Design, on the other hand, provides a complete set of components for a dense data dashboard:

Tables with built-in sort, filter, pagination, and expandable rows (used for the technical details of each find)

Statistics and Cards for the summary block at the top of the page

Tags with preset colors for severity

Descriptions to display key-value evidence without needing to manually adjust the layout

Collapse, Alert, and Progress for health scores

Badges for counts

Almost the entire demo screen is assembled from the pre-made components; he only wrote the logic to join the data. This is a strength of this project, as the time spent learning new things was focused on PowerShell rather than frontend.

A small note: if he wants to "show as much information as possible," tables with expandable rows will be more effective than card-based layouts. Cards look nice but take up space, and each screen only displays a few finds. Tables allow you to see multiple rows at once, filter by severity, and expand to view details when needed. This is also how real audit tools present results.

Prompt 2:

Kèm theo các design sau sẽ bị cấm hoàn toàn.

1. Gradient
2. Glassmorphism
3. Card lồng trong
4. Shadow khắp mọi nơi

Translate:

The following designs are strictly prohibited:

Gradient

Glassmorphism

Card nested within a card

Shadows everywhere

Answer:

Rõ. Bốn thứ đó nằm trong danh sách cấm:

1. Không gradient
2. Không glassmorphism

3. Không card lồng trong card
4. Không shadow rải khắp nơi

Điểm hay là Ant Design mặc định đã đi đúng hướng này rồi. Nó vốn là design system cho dashboard doanh nghiệp, nên mặc định là nền phẳng, viền mảnh, shadow chỉ xuất hiện ở đúng chỗ cần (dropdown, modal, popover, tức là các lớp nổi thật sự). Không có gradient, không có glass. Chỉ cần ông không tự override token màu và shadow của nó là tự khắc tuân thủ.

Cụ thể với bốn điều trên:

- **Gradient và glassmorphism**: không tồn tại trong AntD, chỉ cần không tự thêm CSS là xong
- **Card lồng**: đây là chỗ dễ vi phạm nhất. Khi hiển thị evidence bên trong một finding, phản xạ thường là bọc thêm một Card nữa. Đừng. Dùng Descriptions hoặc Table đặt trực tiếp trong expandable row, không có container bọc ngoài
- **Shadow**: giữ nguyên token mặc định. Nếu dùng Card, đặt variant="borderless" hoặc để mặc định có viền, đừng thêm boxShadow thủ công

Ràng buộc này cũng củng cố thêm lựa chọn Table thay vì card grid mà tao nói ở trên. Table vốn dùng đường kẻ để phân tách, không cần shadow hay container lồng nhau để tạo cảm giác phân tầng.

Translate:

Clearly. Those four things are on the forbidden list:

No gradients

No glassmorphism

No card nesting within cards

No shadows scattered everywhere

The good thing is that Ant Design is already on the right track by default. It's a design system for business dashboards, so by default it has a flat background, thin borders, and shadows only appear where needed (dropdowns, modals, popovers – i.e., truly visible layers). No gradients, no glass. As long as you don't override the color and shadow tokens, it will automatically comply.

Specifically with the four points above:

Gradients and glassmorphism: don't exist in Ant Design; just don't add your own CSS.

Card nesting: this is the easiest violation. When displaying evidence inside a find, the common reaction is to wrap another card. Don't. Use Descriptions or Tables placed directly in expandable rows, without an outer container.

Shadow: keep the default token. If using Card, set variant="borderless" or leave it as the default with a border; don't add a boxShadow manually.

This constraint also reinforces the choice of Table instead of card grid that I mentioned above. Tables use lines to separate elements, and don't need shadows or nested containers to create a sense of layering.

Prompt 3:

Trông có vẻ cơ bản là ổn rồi đấy. Trước hết là cứ build cái frontend trước đi

Translate:

It looks basically fine. Let's build the frontend first.

Answer:

Dashboard đã chạy được, dùng Ant Design toàn bộ, không có component nào tự thiết kế.

Những gì có trong đó:

Khối trên cùng là một Card duy nhất chứa security score (Progress dạng dashboard) và các Statistic đếm theo severity, số object bị ảnh hưởng, số check hoàn thành trên tổng số. Bên dưới là Descriptions bordered hiện toàn bộ metadata của lần scan: domain, DC đã query, functional level, thời gian, module version, tài khoản chạy, scan ID.

Ba tab: Findings, Coverage, Raw JSON.

Tab Findings có search, lọc multi-select theo severity và category, bảng sort được. Mỗi dòng bung ra hiện: Alert giải thích vì sao nguy hiểm, danh sách các bước khắc phục đánh số, bảng affected objects với cột lấy từ keyAttributes trong dictionary, Descriptions cho evidence, link tham khảo, và nút hiện raw JSON của finding đó.

Tab Coverage là thứ tao cho là quan trọng nhất về mặt học thuật. Nó hiển thị riêng những check chạy lỗi, kèm thông báo lỗi. Tao cố tình để trong sample data một check UNCONSTRAINED_DELEGATION bị Access denied, để ông thấy trường hợp đó render ra sao. Có cả Alert cảnh báo ở đầu trang. Đây là chỗ giáo viên hay vặn: nếu tool báo "không tìm thấy vấn đề" mà thực ra là không kiểm tra được thì kết quả sai lệch nghiêm trọng.

Về ràng buộc thiết kế: không gradient, không glass, không shadow tự thêm (chỉ còn shadow mặc định của AntD ở dropdown và popover). Về card lồng, chỗ dễ vi phạm nhất là expanded row, tao để Descriptions và Table nằm trực tiếp trong đó, không bọc Card. Toàn trang chỉ có đúng một Card.

Hai thứ hữu ích về sau: nút “Load scan file” và “Load dictionary” cho phép nạp JSON thật từ máy, nên khi module PowerShell xuất được file thật thì ông thử ngay không cần sửa code. Và bảng affected objects cũng expand được để xem những attribute không nằm trong keyAttributes, nên không có dữ liệu nào bị giấu mất.

Lưu ý khi port sang repo thật: file này là single-file HTML dùng UMD build để chạy được ngay. Khi dựng project React/TS thật, ông chuyển sang import bình thường, tách FindingDetail và Dashboard ra file riêng, và thay hai object SAMPLE_SCAN với DICTIONARY bằng fetch từ file JSON. Logic join theo checkId, sort theo severity, và fallback khi thiếu dictionary entry thì giữ nguyên.

Translate:
The dashboard is working, using Ant Design entirely; no custom-designed components.

What's inside:

The top block is a single Card containing the security score (dashboard-style Progress) and statistics counting by severity, number of affected objects, and number of completed checks out of the total. Below is a bordered Descriptions section showing all the metadata of the scan: domain, DC queried, functional level, time, module version, running account, and scan ID.

Three tabs: Findings, Coverage, and Raw JSON.

The Findings tab has search, multi-select filtering by severity and category, and a sortable table. Each row expands to display: an alert explaining why it's dangerous, a numbered list of remediation steps, a table of affected objects

with columns taken from the keyAttributes in the dictionary, descriptions for the evidence, a reference link, and a button to display the raw JSON of that finding.

The Coverage tab is what I consider the most academically important. It separately displays failed checks, along with error messages. I intentionally included an "Access denied" check in the sample data to show you how that case renders. There's also an alert at the top of the page. This is where teachers often point out problems: if the tool reports "no problem found" when it actually couldn't check, the results are seriously inaccurate.

Regarding design constraints: no gradients, no glass, no added shadows (only AntD's default shadows in dropdowns and popovers). Regarding nested cards, the most common violation is the expanded row; I placed the Descriptions and Table directly within the row, not wrapping the Card. The entire page has only one Card.

Two useful things later on: the "Load scan file" and "Load dictionary" buttons allow loading real JSON from the machine, so when the PowerShell module exports a real file, you can test it immediately without modifying the code. And the affected objects table can also be expanded to see attributes not in keyAttributes, so no data is hidden.

Note when porting to a real repository: this is a single-file HTML file that can be built using UMD to run immediately. When building a real React/TS project, he switched to normal import, separated FindingDetail and Dashboard into separate files, and replaced the two objects SAMPLE_SCAN and DICTIONARY with fetches from the JSON file. The join logic based on checkId, sorting by severity, and the fallback for missing dictionary entries remain unchanged.

The standard JSON schemas used to feed data to the AI in these statements are included in the Github repository:

F26_4495_S2_ChrisN/Implementation/ad-audit-dashboard/CONTRACT.md

And

F26_4495_S2_ChrisN/Implementation/ad-audit-dashboard/public/data/